

Module 2 Notes: Node Anatomy and Pointer Mechanics

1. Big idea of Module 2

Module 2 explains the basic building block of a linked list: the **node**.

In Module 1, you learned why linked lists exist:

Arrays have fixed size, but linked lists can grow and shrink during runtime.

Now Module 2 answers the next question:

What is a linked list actually made of?

The answer is:

nodes + pointers

A linked list is not one big block of memory like an array. It is made of separate small blocks called **nodes**. Each node stores data and the address of the next node.

2. What is a node?

A **node** is one unit of a linked list.

A simple singly linked-list node contains two parts:

[data | next]

Part	Meaning
data	The actual value stored in the node
next	A pointer that stores the address of the next node

Example:

```
[10 | next]
```

This node stores the value `10`, and its `next` part can point to another node.

A full linked list may look like this:

```
[10 | next] -> [20 | next] -> [30 | NULL]
```

This means:

1. The first node stores `10`.
2. The first node points to the second node.
3. The second node stores `20`.
4. The second node points to the third node.
5. The third node stores `30`.
6. The third node points to `NULL`, meaning the list ends.

3. Singly linked list node

A **singly linked list** is a linked list where each node points only forward.

Example:

```
10 -> 20 -> 30 -> NULL
```

Each node knows where the next node is.

But each node does **not** know where the previous node is.

So in a singly linked list:

```
You can move forward easily.  
You cannot move backward directly.
```

4. Node definition in C

In C, a singly linked-list node can be written like this:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Let us break it down.

```
int data;
```

This stores the actual value.

Example:

```
data = 10
```

Then:

```
struct Node *next;
```

This stores the address of another node of the same type.

This is the important part.

`next` is not a normal integer. It is a pointer.

It can point to another `struct Node`.

5. Why is it called self-referential?

This structure is called **self-referential** because it refers to its own type.

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Inside `struct Node`, we have:

```
struct Node *next;
```

That means:

A Node contains a pointer to another Node.

This is allowed because `next` stores only an address.

It does not store a full node inside another full node.

6. Why do we use `struct Node *next` instead of `struct Node next`?

This is wrong:

```
struct Node {  
    int data;  
    struct Node next;  
};
```

Why is it wrong?

Because it means:

A Node contains another full Node.
That Node contains another full Node.
That Node contains another full Node.
And so on forever.

The compiler cannot know how much memory one node needs.

Correct version:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

This works because `next` stores only an address.

So the node has a fixed size:

one integer + one pointer

7. Node definition in C++

In C++, we can write the node like this:

```
struct Node {  
    int data;  
    Node *next;  
};
```

This is very similar to C.

In C++, after defining `Node`, we do not need to write `struct Node *next`; we can simply write:

```
Node *next;
```

We can also define it using a class:

```
class Node {  
public:  
    int data;  
    Node *next;  
};
```

For beginners, the `struct` version is simpler.

8. Node memory picture

Suppose we create one node with data `10`.

Memory picture:

```
[data: 10 | next: NULL]
```

Or in C++:

```
[data: 10 | next: nullptr]
```

This node stores:

```
data = 10  
next = nothing
```

If `next` is `NULL` or `nullptr`, it means there is no next node.

So this node is currently the last node.

9. Creating one node in C++

Example:

```
#include <iostream>  
using namespace std;  
  
struct Node {  
    int data;  
    Node *next;  
};  
  
int main() {  
    Node *p = new Node;  
  
    p->data = 10;  
    p->next = nullptr;  
  
    cout << p->data << endl;  
  
    delete p;  
  
    return 0;  
}
```

What happens here?

```
Node *p = new Node;
```

This creates a new node in heap memory.

`p` stores the address of that node.

Memory picture:

```
Stack:                Heap:
p -----> [ data: ? | next: ? ]
```

Then:

```
p->data = 10;
```

Now the node stores `10`.

```
p -----> [ data: 10 | next: ? ]
```

Then:

```
p->next = nullptr;
```

Now the node does not point to another node.

```
p -----> [ data: 10 | next: nullptr ]
```

Finally:

```
delete p;
```

This releases the heap memory.

10. Creating one node in C

In C, we use `malloc` to create a node.

Example:

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *p;

    p = (struct Node *)malloc(sizeof(struct Node));

    p->data = 10;
    p->next = NULL;

    printf("%d\n", p->data);

    free(p);

    return 0;
}

```

Important memory rule

Language	Create memory	Release memory
C	<code>malloc</code>	<code>free</code>
C++	<code>new</code>	<code>delete</code>

Do not mix them.

Wrong in C++:

```

Node *p = new Node;
free(p); // Wrong

```

Wrong in C:

```

struct Node *p = malloc(sizeof(struct Node));
delete p; // Wrong

```

Use the matching pair.

11. What is a pointer?

A pointer is a variable that stores an address.

Example:

```
Node *p;
```

This means:

```
p can store the address of a Node.
```

Important:

```
Node *p;
```

creates only a pointer.

It does **not** create a node.

To actually create a node in C++, we need:

```
p = new Node;
```

In C, we need:

```
p = (struct Node *)malloc(sizeof(struct Node));
```

12. Pointer vs node

This is a very common beginner confusion.

```
Node *p;
```

This creates a pointer named `p`.

Memory picture:

```
Stack:  
  
p -> unknown address
```

No node has been created yet.

But:

```
Node *p = new Node;
```

creates a node in heap memory and stores its address in `p`.

Memory picture:

```
Stack:                Heap:  
  
p -----> [ data | next ]
```

So:

```
p is not the node.  
p points to the node.
```

13. What does the arrow operator `->` mean?

When we have a pointer to a node, we use `->` to access the node's fields.

Example:

```
p->data
```

means:

```
Go to the node pointed to by p, then access data.
```

Example:

```
p->next
```

means:

Go to the node pointed to by p, then access next.

So this:

```
p->data = 10;  
p->next = nullptr;
```

means:

Put 10 into the data part of the node.
Put nullptr into the next part of the node.

14. Dot operator vs arrow operator

If we have a normal node variable, we use `.`.

Example:

```
Node n;  
n.data = 10;  
n.next = nullptr;
```

But if we have a pointer to a node, we use `->`.

Example:

```
Node *p = new Node;  
p->data = 10;  
p->next = nullptr;
```

Simple rule:

Situation	Operator
Normal object	.
Pointer to object	->

In linked lists, we usually work with pointers, so we usually use `->`.

15. Meaning of `NULL`, `0`, and `nullptr`

A pointer should either point to a valid node or point to nothing.

In C, we commonly use:

`NULL`

Older C++ code may also use:

`NULL`

or:

`0`

Modern C++ uses:

`nullptr`

All of these represent the idea:

This pointer does not point to a valid node.

For modern C++, prefer:

`nullptr`

For C, use:

NULL

16. Why does the last node point to NULL?

A linked list must have a clear ending point.

Example:

10 -> 20 -> 30 -> NULL

The last node is 30 .

Its next pointer points to NULL .

That means:

There is no node after 30.
The list ends here.

Without NULL , we would not know where to stop.

17. What is first or head?

A linked list needs a pointer to the first node.

This pointer is usually called:

first

or:

head

Example:

first
|

```
    v
[10 | next] -> [20 | next] -> [30 | NULL]
```

`first` stores the address of the first node.

The whole list is reached through `first`.

That means:

If you lose `first`, you lose access to the whole list.

18. Why should we not move `first` carelessly?

Suppose we have this list:

```
first
|
v
[10] -> [20] -> [30] -> NULL
```

If we do this:

```
first = first->next;
```

Now `first` points to `20`.

```
[10] -> [20] -> [30] -> NULL
      ^
      |
    first
```

The node containing `10` may become lost if no other pointer is pointing to it.

That is dangerous.

Better approach:

```
Node *p = first;  
p = p->next;
```

Now we move `p`, not `first`.

```
first still points to 10.  
p moves to 20.
```

Beginner rule:

Do not move the only pointer to the first node unless you have saved another pointer.

19. Manually creating a two-node linked list

Goal:

```
10 -> 20 -> NULL
```

C++ code:

```
#include <iostream>  
using namespace std;  
  
struct Node {  
    int data;  
    Node *next;  
};  
  
int main() {  
    Node *first = new Node;  
    Node *second = new Node;  
  
    first->data = 10;  
    first->next = second;  
  
    second->data = 20;  
    second->next = nullptr;  
  
    cout << first->data << endl;  
    cout << first->next->data << endl;
```

```
    delete second;  
    delete first;  
  
    return 0;  
}
```

Memory picture:

```
first -----> [10 | next] ----+  
                                |  
                                v  
second -----> [20 | nullptr]
```

The important link is:

```
first->next = second;
```

This connects the first node to the second node.

20. Understanding `first->next->data`

This expression looks difficult at first:

```
first->next->data
```

Break it into parts.

```
first
```

points to the first node.

```
first -> [10 | next]
```

Then:

```
first->next
```

means the node after the first node.


```
first->next -> [20 | nullptr]
```

Then:

```
first->next->data
```

means the data inside the second node.

So if the list is:

```
10 -> 20 -> NULL
```

then:

```
first->next->data
```

is:

```
20
```

21. Pointer assignment: `q = p`

Assume this list:

```
first
 |
 v
[10] -> [20] -> [30] -> NULL
```

And:

```
Node *p = first;
Node *q = nullptr;
```

At the beginning:

```
p points to 10
q points to nothing
```

Now we do:

```
q = p;
```

This copies the address stored in `p` into `q`.

Now both `p` and `q` point to the same node.

```
first
 |
 v
[10] -> [20] -> [30] -> NULL
 ^
 |
 p
 q
```

Important:

```
q = p does not copy the node.
q = p copies the address.
```

So both pointers refer to the same node.

22. Pointer assignment: `q = p->next`

Assume:

```
[10] -> [20] -> [30] -> NULL
 ^
 |
 p
```

Now we do:

```
q = p->next;
```

`p->next` is the address of the node after `p`.

So `q` points to `20`.

```
[10] -> [20] -> [30] -> NULL
  ^       ^
  |       |
  p       q
```

Meaning:

`q` moves to the node after `p`.

23. Pointer movement: `p = p->next`

Assume:

```
[10] -> [20] -> [30] -> NULL
  ^
  |
  p
```

Now we do:

```
p = p->next;
```

This moves `p` forward by one node.

After:

```
[10] -> [20] -> [30] -> NULL
          ^
          |
          p
```

This is one of the most important linked-list operations.

Traversal is built from repeatedly doing:

```
p = p->next;
```

24. Difference between `q = p->next` and `p = p->next`

These two statements are similar, but they do different things.

```
q = p->next
```

```
q = p->next;
```

This moves `q` to the node after `p`.

`p` stays where it is.

```
p = p->next
```

```
p = p->next;
```

This moves `p` itself to the next node.

So:

Statement	Meaning
<code>q = p</code>	<code>q</code> points to the same node as <code>p</code>
<code>q = p->next</code>	<code>q</code> points to the node after <code>p</code>
<code>p = p->next</code>	<code>p</code> moves forward to the next node

25. Pointer dry run 1

Given list:

```
first -> [8] -> [3] -> [12] -> NULL
```

Code:

```
Node *p = first;  
Node *q = nullptr;  
  
q = p;  
p = p->next;  
q = p->next;
```

Start

```
p -> 8  
q -> nullptr
```

After `q = p`

```
p -> 8  
q -> 8
```

Both point to the node containing `8`.

After `p = p->next`

```
p -> 3  
q -> 8
```

`p` moved forward. `q` stayed at `8`.

After `q = p->next`

```
p -> 3  
q -> 12
```

Because the node after `3` is `12`.

Final result:

```
p points to 3  
q points to 12
```

26. Pointer dry run 2

Given list:

```
first -> [5] -> [9] -> [15] -> NULL
```

Code:

```
Node *p = first;  
Node *q = p;  
  
p = p->next;  
p = p->next;
```

Start

```
p -> 5  
q -> 5
```

After first `p = p->next`

```
p -> 9  
q -> 5
```

After second `p = p->next`

```
p -> 15  
q -> 5
```

Final result:

```
p points to 15  
q points to 5
```

Notice that `q` did not move.

Pointers move only when we assign them a new address.

27. Safe pointer checking

Before using a pointer, check if it is valid.

If a pointer is `NULL` or `nullptr`, it does not point to a real node.

Trying to use `p->data` or `p->next` when `p` is `nullptr` is dangerous.

28. Check if pointer is empty

C++:

```
if (p == nullptr)
```

C:

```
if (p == NULL)
```

Meaning:

```
p does not point to a valid node.
```

29. Check if pointer is valid

C++:

```
if (p != nullptr)
```

C:

```
if (p != NULL)
```

Short form:

```
if (p)
```

Meaning:

p points to a valid node.

30. Check if a node is the last node

C++:

```
if (p->next == nullptr)
```

C:

```
if (p->next == NULL)
```

Meaning:

p points to the last node.

Example:

```
10 -> 20 -> 30 -> NULL
              ^
              |
              p
```

Here:

```
p->next == nullptr
```

is true because 30 is the last node.

31. Check if a node has a next node

C++:


```
if (p->next != nullptr)
```

C:

```
if (p->next != NULL)
```

Meaning:

There is another node after p.

Example:

```
10 -> 20 -> 30 -> NULL
  ^
  |
  p
```

Here:

```
p->next != nullptr
```

is true because there is a node after 10.

32. Important safety rule

Before checking:

```
p->next
```

make sure `p` itself is not `nullptr`.

Dangerous:

```
if (p->next == nullptr) {
    cout << "last node";
}
```

This crashes if `p` is `nullptr`.

Safe:

```
if (p != nullptr && p->next == nullptr) {  
    cout << "last node";  
}
```

First we check:

```
p != nullptr
```

Then we check:

```
p->next == nullptr
```

33. Dangerous mistake: using `p->next` when `p` is null

Wrong:

```
Node *p = nullptr;  
p = p->next;
```

Why is this wrong?

Because `p` points to nothing.

So this part:

```
p->next
```

means:

```
Go to nothing and access next.
```

That is invalid.

Correct:

```
if (p != nullptr) {  
    p = p->next;  
}
```

34. Function to check pointer status

C++ example:

```
void checkPointer(Node *p) {  
    if (p == nullptr) {  
        cout << "p is nullptr" << endl;  
    }  
    else if (p->next == nullptr) {  
        cout << "p is valid and it is the last node" << endl;  
    }  
    else {  
        cout << "p is valid and it has a next node" << endl;  
    }  
}
```

Example list

10 -> 20 -> NULL

If `p` points to `10`, output is:

p is valid and it has a next node

If `p` points to `20`, output is:

p is valid and it is the last node

If `p` is `nullptr`, output is:

p is nullptr

35. Full C++ example for Module 2

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node *next;
};

void checkPointer(Node *p) {
    if (p == nullptr) {
        cout << "p is nullptr" << endl;
    }
    else if (p->next == nullptr) {
        cout << "p is valid and it is the last node" << endl;
    }
    else {
        cout << "p is valid and it has a next node" << endl;
    }
}

int main() {
    Node *first = new Node;
    Node *second = new Node;

    first->data = 10;
    first->next = second;

    second->data = 20;
    second->next = nullptr;

    Node *p = first;
    Node *q = nullptr;

    cout << "first node data: " << p->data << endl;

    q = p->next;
    cout << "second node data: " << q->data << endl;

    checkPointer(p);
    checkPointer(q);
    checkPointer(nullptr);

    delete second;
    delete first;
}
```

```
    return 0;
}
```

36. Full C example for Module 2

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void checkPointer(struct Node *p) {
    if (p == NULL) {
        printf("p is NULL\n");
    }
    else if (p->next == NULL) {
        printf("p is valid and it is the last node\n");
    }
    else {
        printf("p is valid and it has a next node\n");
    }
}

int main() {
    struct Node *first;
    struct Node *second;

    first = (struct Node *)malloc(sizeof(struct Node));
    second = (struct Node *)malloc(sizeof(struct Node));

    first->data = 10;
    first->next = second;

    second->data = 20;
    second->next = NULL;

    printf("first node data: %d\n", first->data);
    printf("second node data: %d\n", first->next->data);

    checkPointer(first);
    checkPointer(second);
}
```

```

    checkPointer(NULL);

    free(second);
    free(first);

    return 0;
}

```

37. Boundary conditions

A **boundary condition** checks special positions or special cases.

For linked lists, important boundary checks are:

Condition	Meaning
<code>p == NULL</code> or <code>p == nullptr</code>	<code>p</code> points to nothing
<code>p != NULL</code> or <code>p != nullptr</code>	<code>p</code> points to a valid node
<code>if (p)</code>	Short form for checking if <code>p</code> is valid
<code>p->next == NULL</code> or <code>p->next == nullptr</code>	<code>p</code> is the last node
<code>p->next != NULL</code> or <code>p->next != nullptr</code>	<code>p</code> has a next node

These checks prevent crashes and help us know where we are in the list.

38. Complexity of Module 2 operations

Most single pointer operations are constant time.

Operation	Meaning	Time complexity
<code>q = p</code>	Copy an address	<code>O(1)</code>
<code>q = p->next</code>	Copy the address of the next node	<code>O(1)</code>
<code>p = p->next</code>	Move one step forward	<code>O(1)</code>
<code>p == nullptr</code>	Check if pointer is empty	<code>O(1)</code>
<code>p != nullptr</code>	Check if pointer is valid	<code>O(1)</code>
<code>p->next == nullptr</code>	Check if node is last	<code>O(1)</code>

One pointer movement is $O(1)$.

But repeated movement through many nodes becomes $O(n)$.

Example:

```
while (p != nullptr) {  
    p = p->next;  
}
```

If the list has n nodes, this loop visits up to n nodes.

So the loop is:

$O(n)$

Important idea:

The cost does not come from one pointer move.
The cost comes from repeating pointer moves many times.

39. Doubly linked node preview

Module 2 mainly focuses on singly linked nodes.

But later, you will learn about **doubly linked lists**.

A singly linked node has:

[data | next]

A doubly linked node has:

[prev | data | next]

Example:

```
NULL <- [10] <-> [20] <-> [30] -> NULL
```

In a doubly linked list:

Pointer	Meaning
<code>prev</code>	Address of the previous node
<code>next</code>	Address of the next node

So a doubly linked node can move both forward and backward.

C-style preview:

```
struct Node {  
    struct Node *prev;  
    int data;  
    struct Node *next;  
};
```

You do not need to master doubly linked lists yet.

For now, just remember:

```
Singly node: data + next  
Doubly node: prev + data + next
```

40. Common beginner mistakes

Mistake 1: Thinking `Node *p;` **creates a node**

Wrong idea:

```
Node *p;
```

means a node has been created.

Correct idea:


```
Node *p;
```

creates only a pointer.

A node is created only when we allocate memory:

```
p = new Node;
```

or in C:

```
p = malloc(sizeof(struct Node));
```

Mistake 2: Thinking `q = p` copies the full node

Wrong idea:

```
q = p;
```

creates a copy of the node.

Correct idea:

```
q = p;
```

copies the address.

Both pointers point to the same node.

Mistake 3: Moving `first` instead of a temporary pointer

Bad:

```
first = first->next;
```

This may lose the first node.

Better:

```
Node *p = first;  
p = p->next;
```

Move `p`, not `first`.

Mistake 4: Accessing `p->next` before checking `p`

Bad:

```
if (p->next == nullptr) {  
    cout << "last";  
}
```

If `p` is `nullptr`, this is invalid.

Better:

```
if (p != nullptr && p->next == nullptr) {  
    cout << "last";  
}
```

Mistake 5: Mixing C and C++ memory functions

Bad:

```
Node *p = new Node;  
free(p);
```

Bad:

```
struct Node *p = malloc(sizeof(struct Node));  
delete p;
```

Correct:

```
Node *p = new Node;  
delete p;
```

Correct:

```
struct Node *p = malloc(sizeof(struct Node));  
free(p);
```

41. Mini summary

Module 2 teaches the real foundation of linked lists.

The key idea is:

A linked list is made of nodes.
Each node stores data and a pointer to the next node.

Important terms:

Term	Meaning
Node	One block in a linked list
Data	The value stored in a node
Next	Pointer to the next node
Pointer	Variable that stores an address
<code>first</code> / <code>head</code>	Pointer to the first node
<code>NULL</code> / <code>nullptr</code>	No valid node address
Self-referential structure	A structure that contains a pointer to the same structure type

Important pointer statements:

Statement	Meaning
<code>q = p</code>	<code>q</code> points to the same node as <code>p</code>
<code>q = p->next</code>	<code>q</code> points to the node after <code>p</code>
<code>p = p->next</code>	<code>p</code> moves forward to the next node

Important safety rule:

Do not use `p->data` or `p->next` unless `p` points to a valid node.

42. Practice questions

Question 1

What are the two parts of a singly linked-list node?

Answer

A singly linked-list node has:

`data` + `next`

`data` stores the value.

`next` stores the address of the next node.

Question 2

Why is this structure self-referential?

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

Answer

Because `next` is a pointer to another `struct Node`.

The structure refers to its own type.

Question 3

What is the difference between `q = p` and `q = p->next`?

Answer

```
q = p;
```

makes `q` point to the same node as `p`.

```
q = p->next;
```

makes `q` point to the node after `p`.

Question 4

Why can `p = p->next` be dangerous?

Answer

If `p` is the only pointer to the first node, moving `p` forward may lose access to the original first node.

Also, if `p` is `nullptr`, then `p->next` is invalid.

Question 5

Which condition checks whether a node is the last node?

Answer

C++:

```
p->next == nullptr
```

C:

```
p->next == NULL
```

But first make sure `p` itself is valid.

Safe C++ version:

```
p != nullptr && p->next == nullptr
```

Question 6

What does `first` or `head` store?

Answer

It stores the address of the first node in the linked list.

If we lose `first` or `head`, we may lose access to the whole list.

Question 7

What does `NULL` or `nullptr` mean?

Answer

It means the pointer does not point to a valid node.

In a linear singly linked list, the last node points to `NULL` or `nullptr`.

Question 8

What is the time complexity of one pointer assignment like `q = p`?

Answer

It is $O(1)$ because it copies one address.

Question 9

What is the time complexity of moving through all nodes using `p = p->next` in a loop?

Answer

It is $O(n)$ because the loop may visit every node.

43. Final Module 2 summary

Module 2 is about understanding nodes and pointer movement.

A node is:

```
[data | next]
```

A linked list is:

```
first -> [10] -> [20] -> [30] -> NULL
```

The most important pointer ideas are:

```
q = p
```

copies an address.

```
q = p->next
```

makes `q` point to the node after `p`.

```
p = p->next
```

moves `p` forward.

The most important safety idea is:

```
Never use p->data or p->next if p is NULL or nullptr.
```

The most important linked-list pointer is:

```
first / head
```

because it gives access to the whole list.

Once you understand Module 2, you are ready for Module 3: traversal, display, and creating a linked list from an array.